

Embedding Models for Fine-Grained Visual Homoglyph Detection

Seton Liu ^{1,†} , Hailey Boriel ^{1,†} , Valerie Xin ¹ , Rafael Palacios ^{2,3} , Miguel Domingos de Santana Wanderley ⁴ ,
Guilherme Rinaldo ⁴  and Amar Gupta ^{1,5,*} 

¹ Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, 77 Massachusetts Ave., Cambridge, MA 02139, USA; setliu22@mit.edu (S.L.); hail01@mit.edu (H.B.); valxin@mit.edu (V.X.); agupta@mit.edu (A.G.)

² Cybersecurity at MIT Sloan (CAMS), Massachusetts Institute of Technology, 77 Massachusetts Ave., Cambridge, MA 02139, USA; palacios@mit.edu (R.P.)

³ Institute for Research in Technology, Universidad Pontificia Comillas, Alberto Aguilera 23, 28015 Madrid, Spain

⁴ Instituto de Ciência e Tecnologia Itaú - ICTi, Av. Engenheiro Armando de Arruda Pereira 774, Jabaquara - São Paulo, São Paulo 04308-000, Brazil; miguel.wanderley@itau-unibanco.com.br (M.D.S.W.); guilherme.rinaldo@itau-unibanco.com.br (G.R.)

⁵ AI Institute for Community-Engaged Research (AI-ICER), The University of Texas at El Paso, 500 West University Avenue, El Paso, TX 79968, USA

* Correspondence: agupta@mit.edu

† These authors contributed equally to this work.

Abstract

Homoglyph attacks are a form of spoofing used by attackers to deceive users by exploiting visually similar characters. This paper presents a visual comparison framework in which each name is rendered as an image and processed as an ordered sequence of vertical slices. Seven neural architecture families are compared on the Woodbridge benchmark and a novel synthetic substitution dataset. Both selected models first use a shared Conv1D frontend to extract features from the ordered image slices. For the Woodbridge benchmark, the selected Interaction CNN compares every slice of one name with every slice of the other to create a visual interaction map, which is then processed by a CNN for classification. This model achieves a test ROC-AUC of 0.9942, accuracy of 0.9634, and F1 of 0.9715. For the synthetic substitution dataset, the selected model instead uses two bidirectional cross-attention blocks, allowing the two names to exchange information before classification. It achieves a test ROC-AUC of 0.99999, accuracy of 0.9994, and F1 of 0.9995. The Siamese Conv2D baseline outperforms the model that only slices the names and processes them with a Conv1D on both datasets, showing that slicing and Conv1D processing alone do not account for the strongest results.

Keywords: Homoglyph detection; Unicode spoofing; visual similarity; Siamese networks; visual encoding

Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Mach. Learn. Knowl. Extr.*

for possible open access publication

under the terms and conditions of the

[Creative Commons Attribution](#)

(CC BY) license.

1. Introduction

Homoglyph attacks through business-name spoofing remain a persistent threat within the broader phishing landscape. The Anti-Phishing Working Group recorded 1,003,924 phishing attacks in the first quarter of 2025, with software-as-a-service and webmail providers, financial institutions, and online payment companies collectively accounting for nearly half of reported attacks [1]. In a homoglyph attack, visual similarity is the central

mechanism of deception. A single Unicode replacement, such as a Cyrillic a for a Latin a or the digit 0 for the letter o, can create an identifier that appears nearly identical to a human reader while representing a different character sequence.

The effectiveness of this technique is grounded in the importance of visual appearance in human trust decisions. Dhamija, Tygar, and Hearst found that good phishing websites fooled 90% of participants even when the study primed participants to look for spoofs, and that 23% did not look at the address bar, status bar, or security indicators [2]. The size of the Unicode character space further complicates manual defenses. Roshanbin and Miller demonstrated that, even when attention is restricted to homoglyphs of individual Latin letters, `http://www.google.com` can be rendered in more than 1,360 visually deceptive ways [3]. Together, these findings indicate that effective detection systems should account for rendered visual appearance rather than relying only on symbolic representations such as code points or byte sequences.

Traditional methods for homoglyph and typosquatting detection commonly use Levenshtein and Damerau–Levenshtein distance, which treat visually similar and unrelated substitutions alike [4,5]. TypoPegging adds position-weighted costs and a curated visual confusion matrix, but substitutions outside that table receive no specialized visual treatment [6]. Text embeddings such as Word2Vec and Bidirectional Encoder Representations from Transformers (BERT) capture semantic or contextual relationships rather than rendered character shape [7,8]. These limitations motivate direct modeling of visual appearance.

Image-based approaches instead model rendered similarity directly. Woodbridge et al. used a Siamese convolutional neural network (CNN) over rendered names, GlyphNet applied an attention-based CNN to rendered domains, and Ginsberg and Yu compared spatial zones within individual glyphs [9–11]. These studies establish the value of visual evidence, but whole-image and glyph-level representations do not explicitly provide a learned left-to-right sequence of local comparisons across a full name.

This paper addresses the remaining gap through a visual comparison framework that renders each name as a fixed-height grayscale image and represents it as an ordered sequence of vertical slices. A shared one-dimensional convolutional (Conv1D) frontend produces slice-level features, after which the model may encode each name independently, exchange information through cross-attention, or construct a dense all-pairs interaction map. The work makes four contributions:

1. a controlled comparison of seven visual architecture families under a common validation-only model-selection protocol;
2. a regenerated synthetic substitution benchmark with visually recognizable positives and nearest-neighbor hard negatives;
3. two complementary visual-substitution inventories: strict raster-based lookalikes and OCR-confusable glyphs screened for visual similarity and recognizability; and
4. test evaluation of one validation-selected architecture for each dataset.

2. Related Work

2.1. String Similarity Methods

The dominant baselines for detecting homoglyph attacks and typosquatting are string edit-distance metrics, most notably Levenshtein distance [4]. Levenshtein distance measures the minimum number of single-character insertions, deletions, and substitutions required to transform one string into another. Although it is widely used because it is simple and interpretable, its flat-cost assumption treats visually similar and visually dissimilar substitutions the same. Replacing a character with a nearly identical homoglyph is therefore penalized no differently from replacing it with an unrelated character.

Table 1. Summary of string-similarity methods used for homoglyph and typosquatting detection.

Method	Core mechanism	Primary limitation for homoglyph de- tection
Levenshtein distance [4]	Minimum insertions, deletions, and substitutions needed to transform one string into another	Flat substitution cost gives nearly identical and unrelated character pairs the same unit penalty
Damerau–Levenshtein [5]	Adds adjacent-character transposition as a fourth edit operation	Retains the flat substitution cost and therefore does not model rendered character structure
TypoPegging [6]	Position-weighted edit distance with an expert-curated matrix of visual confusion scores	Coverage is bounded by the predefined confusion table, and a threshold may not distinguish intentional spoofs from legitimately similar names

Damerau–Levenshtein distance extends Levenshtein distance by adding transposition of adjacent characters as a fourth edit operation [5]. This extension captures another common spoofing pattern, but retains the flat substitution cost and therefore inherits the same blindness as Levenshtein distance.

A notable attempt to address this limitation is TypoPegging [6], which modifies edit distance using a position-decay term and a visual-distance matrix containing expert-estimated confusion scores for character pairs. The method achieved an F1 score of 94.45% on 1,495 real typo domains. However, its threshold cannot always distinguish an intentional spoof from a legitimately similar registered name, and its coverage is limited to its manually curated confusion matrix. Novel Unicode substitutions absent from that matrix receive no specialized visual treatment. Table 1 summarizes these methods and limitations.

2.2. Image-Based Homoglyph Detection

The limitations of string-based methods motivated a shift toward visual representations of rendered text. An early example is the hit-zone framework of Ginsberg and Yu, which renders glyphs as images and encodes them into four binary zone maps at progressively finer spatial granularities [11]. Similarity is then computed over aligned zone vectors, allowing visually matching cross-script characters to be compared independently of their Unicode representations.

A foundational learned approach is the Siamese neural network by Koch, Zemel, and Salakhutdinov [12]. Twin convolutional branches with tied weights process two input images and produce representations that can be compared for similarity. Woodbridge et al. adapted this approach to rendered process and domain names, training a lightweight Siamese CNN with contrastive loss [9]. Because each name is encoded independently, reference embeddings can be precomputed for retrieval.

Gupta, Tomar, and Garg identified dataset scale and the requirement for paired real-spoof inputs as limitations of the Woodbridge approach [10]. They proposed GlyphNet, an attention-based CNN trained on a large constructed dataset of rendered domains. By treating homoglyph detection as image classification, GlyphNet removes the need for a legitimate comparison name during inference. Lee et al. addressed data scarcity from the generative side with PhishGAN, a conditional generative adversarial network that synthesizes homoglyph images from rendered text inputs [13]. Homoglyph substitutions can also affect downstream multimodal systems; Struppek et al. demonstrate cross-script effects in text-to-image synthesis [14].

Woodbridge et al., GlyphNet, and PhishGAN process each rendered name as a whole image, whereas the hit-zone framework operates on individual glyphs. These methods do not explicitly decompose a full rendered name into a positionally ordered sequence of column-level representations. Consequently, they provide limited direct support for full-name, position-indexed analysis of where a visual discrepancy occurs.

2.3. Sequential Visual Encoding

The key visual design decision in this paper is to decompose a rendered name into a left-to-right sequence of narrow vertical slices and process that sequence with convolutional, recurrent, or attention-based components. This choice is motivated by prior work establishing the viability of column-oriented image structure as a unit of sequential analysis.

The closest architectural precedent is the convolutional recurrent neural network (CRNN) by Shi, Bai, and Yao [15]. A convolutional image frontend produces a feature map that is read column by column from left to right, yielding an ordered sequence of visual feature vectors. Bidirectional recurrent layers then incorporate context from both directions. The architecture demonstrates that horizontal position in a fixed-height text image naturally corresponds to position in a visual sequence. The present work applies the same spatial inductive bias to pairwise visual comparison.

More recent sequential vision architectures further demonstrate that axis-aligned recurrent processing can be competitive for visual tasks. Sequencer, for example, applies long short-term memory layers along image axes rather than relying exclusively on convolution or self-attention [16]. Together, these precedents provide a principled basis for representing a rendered name as ordered vertical slices.

2.4. Pairwise Similarity and Interaction Models

Independently encoded Siamese models, following the visual-verification formulation by Koch, Zemel, and Salakhutdinov, are efficient because reference embeddings may be precomputed [12]. Their two inputs, however, do not interact until the final comparison.

Cross-attention permits each position in one name to query the other name before pooling, providing an explicit mechanism for repeated pairwise exchange. Interaction-map models take a different approach by forming an all-pairs similarity matrix and processing it as an image. MatchPyramid is a direct precedent for learning hierarchical patterns over such a matrix [17]. The Interaction CNN evaluated here extends that idea to visual slice features and uses multiple interaction channels.

2.5. Text Embeddings and Their Limitations for Visual Similarity

Word2Vec and BERT are optimized primarily for semantic or contextual similarity rather than rendered appearance [7,8]. Names may therefore receive similar representations because they occur in similar contexts even when their glyphs differ, while rare Unicode substitutions may not receive meaningful representations. This objective is misaligned with homoglyph detection, where character appearance is the primary signal.

Prior work therefore leaves a gap between symbolic methods that do not directly model appearance and visual methods that do not learn position-indexed comparisons across a full name. The present framework addresses this gap with ordered slice encoders and explicit pairwise interaction mechanisms.

3. Materials and Methods

3.1. Problem Formulation

Let s_a and s_b denote a candidate name and a reference legitimate name. The task is to predict $y \in \{0, 1\}$, where $y = 1$ indicates that the candidate is a visual spoof of the reference.

Independently encoded models apply a shared encoder $f(\cdot)$ to both names and compare the resulting vectors through a symmetric head. Pairwise models retain the two slice-level sequences and permit direct interaction before classification. Each model outputs a single scalar logit for each pair. The logit is converted to a spoof probability through a sigmoid function, and the model is trained against the binary label using binary cross-entropy with logits.

3.2. Datasets

Woodbridge benchmark.

The datasets used for the Woodbridge benchmark are derived from the domain-name and process-name data introduced by Woodbridge et al. [9]. In these datasets each row contains `fraudulent_name`, `real_name`, and `label`; the training, validation, and test splits contain 976,122, 9,999, and 256,886 pairs. Some negative pairs are trivially distinguishable using potentially irrelevant patterns such as string length, so strong performance on this benchmark alone does not demonstrate that models have learned to use fine-grained visual similarity to correctly classify spoofs.

Synthetic substitution dataset.

The synthetic benchmark retains the same pair-classification task but regenerates the positive class from controlled visual substitutions and replaces distant random negatives with nearest-neighbor hard negatives. Its training, validation, and test splits contain 976,122, 9,999, and 256,886 pairs. Reference names do not overlap across the training, validation, and test splits, so each reference name and its generated positive spoof variants are confined to a single split.

3.3. Synthetic Substitution Dataset Construction

3.3.1. Motivation and Design Objective

The synthetic substitution dataset is designed to contain positive pairs that remain visually recognizable while becoming less separable using string features such as edit distance and string length alone. Candidate generation strategies are compared using three criteria:

$$\max \mathbb{E}[\text{LEGIT}], \quad \min \text{AUC}_{\text{raw}}, \quad \min \text{AUC}_{\text{OCR}}. \quad (1)$$

Here, the expected LEGIT score is taken over generated positive pairs only, while AUC_{raw} and AUC_{OCR} denote the ROC-AUC of diagnostic classifiers using raw and OCR-normalized string features, respectively. The criteria are partly contradictory. Substitutions that are very similar visually to the original character are more likely to be mapped by optical character recognition (OCR) to the source character and produce an easily detected near-zero normalized distance (thereby making the pair easier to distinguish using OCR-normalized string features). Replacements that survive OCR normalization can weaken this signal, but may also make the spoofed name less visually similar to the legitimate name and therefore less likely to deceive a human reader.

3.3.2. LEGIT Visual Recognizability Score

Seth et al. introduced LEGIT, a human-annotated dataset for measuring the legibility of visually perturbed text, and trained models to predict how legible these perturbations are [18]. We use their released vision-based model to measure the recognizability of the generated positive examples. Higher scores indicate that the fraudulent name is more recognizable as the intended legitimate name. LEGIT is calculated only for positive pairs. Scoring follows the released model's rendering convention rather than the DejaVu Sans renderer used by the detection architectures: the fraudulent name, two spaces, and the

legitimate name are rendered together in Unifont at 32 pixels as black text on a white 40-pixel-high image. Table 2 gives two illustrative scores.

Table 2. Illustrative LEGIT scores. Higher values indicate greater predicted recognizability of the fraudulent name as the corresponding legitimate name.

Real name	Fraudulent name	LEGIT
cuny	cqnfy	−1.5823
theorchard	thaorchard	4.8148

3.3.3. Visual Substitution Inventories

The final synthetic generator uses two complementary sources of single-character substitutions: OCR-confusable glyphs and strict exact lookalikes. Both are based on glyphs rendered in DejaVu Sans, but they capture different types of visual similarity.

OCR-confusable inventory.

Candidates are generated for lowercase letters, digits, and the hyphen by comparing each source character with Unicode glyphs available in DejaVu Sans. Visual similarity is measured from the rendered glyphs using pixel-level cosine similarity. The most visually similar candidates are then evaluated using the released TrOCR printed and handwritten checkpoints [19]. Candidates that remain visually similar to the source character but are recognized by OCR as a different character are manually reviewed, and visually convincing replacements are retained.

Exact-lookalike inventory.

We retain exact-lookalike replacements with a glyph-similarity score of at least 0.999 and nearly the same rendered size and thickness as the source character. These substitutions are intended to capture glyphs that appear almost identical to the source character in DejaVu Sans.

3.3.4. Positive Example Regeneration

Positive examples pair a generated visually deceptive name with the legitimate name it imitates, with all legitimate names drawn from the Woodbridge benchmark. Eligible operations include OCR-confusable substitutions, exact-lookalike substitutions, and a small set of multicharacter substitutions, such as m to rn.

For each positive pair, the generator first selects a target Levenshtein distance. It then generates possible spoofed names using the allowed visual substitutions and prioritizes candidates that most closely match the target distance. Among these candidates, OCR-confusable substitutions are preferred, followed by exact lookalikes and then multicharacter substitutions. If the preferred candidate has already been used, the generator selects the next available option.

Because some multicharacter substitutions change the length of the name, a single substitution can contribute more than one unit of Levenshtein distance. We evaluate three target-distance strategies:

- **Distance-1 target:** prefer generated names at Levenshtein distance 1;
- **Distance-2 target:** prefer generated names at Levenshtein distance 2; and
- **Mixed 65% distance-2 target (mix65):** target distance 2 with probability 0.65 and distance 1 otherwise.

In the final mix65 validation split, 2,198 positive pairs have Levenshtein distance 1 and 4,230 have distance 2. A small number of distance-3 and distance-4 examples appear

Table 3. Validation-set generation study. Higher LEGIT statistics indicate greater recognizability; lower random-forest ROC-AUC indicates reduced symbolic separability.

Version	Rows	Pos.	Neg.	LEGIT mean	LEGIT Q25	LEGIT median	Raw RF AUC	OCR-table RF AUC
Woodbridge Validation	9,999	6,428	3,571	2.2946	1.3023	2.5630	0.9684	0.9874
Distance-1 target	9,999	6,428	3,571	3.8569	3.5272	4.2412	0.9480	0.9691
Distance-2 target	9,999	6,428	3,571	3.3678	2.7478	3.7852	0.8237	0.8901
Mixed 65% distance-2 target	9,999	6,428	3,571	3.5425	2.9701	3.9745	0.8621	0.9190

in the larger training and test splits because some multicharacter substitutions contribute more than one edit or because candidates at the preferred distance were unavailable.

3.3.5. Hard Negative Construction

Randomly pairing unrelated names creates trivial negatives that often differ greatly in length and composition. Our synthetic substitution dataset instead pairs each reference with a different legitimate name drawn from the Woodbridge benchmark. For final mix65 split generation, the target negative Levenshtein distance is independently sampled as 2 with probability 0.65 and 1 otherwise. The generator searches length-indexed candidate pools, selects a name at the requested distance when available, and otherwise uses the nearest available candidate. This construction reduces the gap in average edit distance between the two classes and makes edit distance less useful as a trivial classification cue.

3.3.6. Validation-Set Generation Study

The three positive strategies are evaluated by regenerating the complete 9,999-row validation set, including 6,428 positives and 3,571 negatives. Two diagnostic random-forest (RF) classifiers use distance features from the raw strings and from OCR-normalized strings. Higher LEGIT values are desirable, while lower diagnostic ROC-AUC indicates less separability using distance features.

In Table 3, the distance-1 target produces the strongest LEGIT scores but remains highly separable using text distance features. The distance-2 target most strongly reduces the ROC-AUC of classifiers trained on text distance features but lowers LEGIT scores. The mix65 strategy lies between these extremes and is selected as a final compromise.

3.3.7. Final Dataset Construction

The mix65 strategy generates the final training, validation, and test splits while preserving the source split sizes and class distributions. Generated positive fraudulent names are unique after Unicode NFKC and case normalization across all three splits. Negative pair tuples are also unique, although the same legitimate candidate string may appear in more than one negative pair. Reference names in the `real_name` column do not cross splits, and the resulting split contents are frozen before architecture selection.

3.4. Visual Representation and Architectures

3.4.1. Rendering and Vertical Slicing

Each name is rendered in DejaVu Sans as a fixed-height image with white text on a black background while preserving its original width. For models that use image slices, the image is divided into narrow vertical strips from left to right. Each strip contains a small portion of the rendered name, allowing the model to learn local, position-specific visual patterns. If the final strip is shorter than the selected strip width, it is padded, and the padded region is ignored during later processing. Figure 1 summarizes the common processing pipeline.

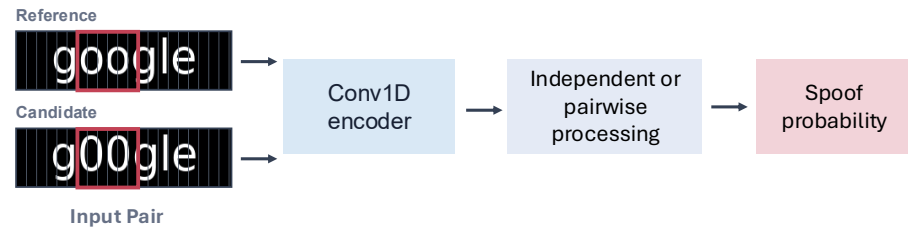


Figure 1. Overview of the general architecture.

3.4.2. Shared Conv1D Frontend

All sliced architectures begin with the same Conv1D frontend. Each vertical strip is converted into a feature vector, and the Conv1D processes these features in their left-to-right order. This produces a sequence of learned visual features representing different positions in the name. The Conv1D frontend is trained jointly with the rest of each model.

3.4.3. Architecture Families

Conv1D.

The simplest sliced model uses only the shared Conv1D frontend. The sequence of slice features is combined into a single vector representing each name. The resulting candidate and reference vectors are then compared to make the final prediction.

Conv1D + BiLSTM.

The BiLSTM processes the sequence of slices in both directions. This allows each slice to incorporate information from visual content appearing both before and after it in the name. The candidate and reference names are still processed independently before their final vectors are compared.

Conv1D + Transformer.

The Transformer also processes each name independently, but self-attention allows each slice to incorporate information from other positions in the same name [20]. Rotary Position Embeddings preserve information about slice position [21]. We evaluate both one-layer and two-layer Transformer variants.

One-block and two-block cross-attention.

The cross-attention models allow the candidate and reference names to interact before the final prediction. In each block, slices from the candidate name attend to slices from the reference name, while slices from the reference name also attend to the candidate. This allows information from one name to directly influence the representation of the other. The two-block model repeats this exchange a second time before pooling and classification.

Interaction CNN.

The Interaction CNN compares every encoded slice of the candidate name with every encoded slice of the reference name. These comparisons form a two-dimensional interaction map, where each location represents the relationship between one position in the candidate and one position in the reference. The model can use either a cosine-similarity map or a four-channel map containing cosine similarity, scaled dot product, negative mean absolute difference, and mean elementwise product. A Conv2D network then processes the map to learn larger patterns in how the two names visually align. This design is related to MatchPyramid [17].

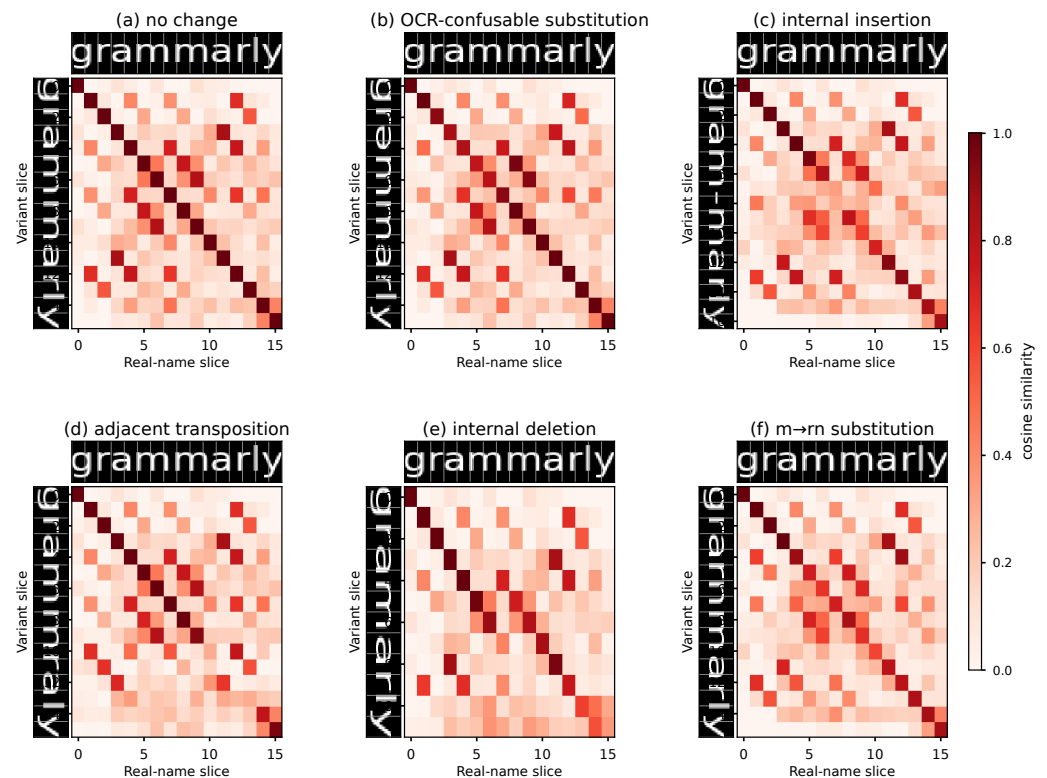


Figure 2. Illustrative slice-to-slice cosine-similarity maps for the reference name *grammarly* and six candidate renderings. Columns correspond to slices of the real name and rows correspond to slices of the candidate name, with darker cells indicating greater similarity. Each substitution type produces a distinct pattern in the interaction map.

Figure 2 shows examples of the slice-to-slice cosine-similarity maps used as one possible Interaction CNN input.

Whole-image CNN.

The Siamese Conv2D baseline does not divide names into vertical strips. Instead, the complete candidate and reference images are processed independently using the same Conv2D encoder. This produces one vector for each name, and the two vectors are then compared to make the final prediction. This provides a whole-image baseline for evaluating whether slicing and Conv1D processing alone improve performance.

3.4.4. Pooling and Pair Scoring

Most sliced architectures produce a sequence of feature vectors, with each vector representing a different position in the rendered name. Before the candidate and reference can be compared, these slice-level features are combined into a single vector for each name. We evaluate mean pooling, max pooling, and learned attention pooling. Mean pooling averages information across the slices, max pooling keeps the strongest value for each learned feature, and attention pooling learns which slices should contribute most strongly to the final vector.

For architectures that encode the candidate and reference independently, their final vectors can be compared directly using cosine similarity or passed to a learned comparison head. Cosine similarity directly measures the similarity between the two vectors. Alternatively, a linear layer or multilayer perceptron (MLP) can learn how differences between the two vectors relate to whether the names form a spoof pair. These learned heads use the absolute difference and elementwise product of the two vectors as input. A calibrated cosine-similarity variant is also evaluated.

3.5. Architecture Selection and Evaluation333

3.5.1. Validation-Only Search Protocol334

Initial experiments were used to determine reasonable ranges for hyperparameters such as learning rate, batch size, dropout, and model size. After these ranges were established, five final hyperparameter configurations were evaluated for each architecture family on each dataset. Each configuration was trained for at most five epochs, and the configuration with the highest validation ROC-AUC was selected.

After the best architecture and hyperparameter configuration were selected for each dataset, that configuration was trained for 25 epochs on the full training split. Validation performance was measured after each epoch, and the model from the epoch with the highest validation ROC-AUC was retained. This final model was then evaluated once on the test set. The test set was not used for architecture selection, hyperparameter selection, or training.

3.5.2. Implementation and Metrics346

Experiments were run on NVIDIA GPU nodes of the MIT OpenMind Research Computing and Data cluster through Slurm. Python, NumPy, PyTorch, and CUDA random states use seed 7, and deterministic cuDNN execution is enabled. Final reporting includes ROC-AUC, accuracy, precision, recall, F1, and Matthews correlation coefficient at a threshold of 0.5000. The test set is not used to select the classification threshold.

3.6. Use of Generative AI Tools352

ChatGPT (OpenAI) was used to assist with reviewing manuscript text for grammatical and logical consistency and to suggest rewrites of sections identified as unclear or incorrect. Codex (OpenAI) was used to assist with coding and script generation. All AI-assisted text and code were reviewed, verified, and modified as necessary by the authors.

4. Results357

4.1. Five-Epoch Validation Screening358

Table 4 reports the best validation ROC-AUC obtained for each architecture family during the five-trial screening process.

Table 4. Best validation ROC-AUC from the five-trial screening studies, with each trial trained for at most five epochs.

Architecture	Woodbridge	Synthetic
Conv1D	0.9774	0.9987
Conv1D + BiLSTM	0.9944	0.9999
Conv1D + Transformer	0.9944	0.9994
One-block cross-attention	0.9950	0.9997
Two-block cross-attention	0.9955	1.0000
Interaction CNN	0.9963	0.9998
Whole-image CNN	0.9914	0.9999

The synthetic two-block cross-attention ROC-AUC shown as 1.0000 is rounded from 0.999960. The whole-image CNN outperforms the plain sliced Conv1D on both datasets. All architectures on the synthetic dataset achieve validation ROC-AUC above 0.998, while all Woodbridge architectures except the plain Conv1D exceed 0.9900. The particularly high performance of the visual models on the synthetic dataset is expected because the dataset was specifically designed to make string-distance features less useful while making visual similarity the main signal for distinguishing positive and negative pairs.

4.2. Selected Architectures and Hyperparameters368

Table 5 lists the validation-selected configurations used for final training.

Table 5. Selected final configuration for the model trained on each dataset.

Setting	Woodbridge benchmark	Synthetic substitution dataset
Architecture	Conv1D + Interaction CNN	Conv1D + two-block cross-attention
Rendering and slicing	32-px image; 16-px strips; stride 8	32-px image; 6-px non-overlapping strips
Shared Conv1D	2 layers; kernel 3; 128-dimensional features	2 layers; kernel 3; 128-dimensional features
Pair processor	Four-channel interaction map; 3-layer Conv2D	2 bidirectional cross-attention blocks; 4 heads
Pooling and classifier	Masked mean pooling; 32-unit MLP	Attention pooling; linear comparison head
Learning rate	2.2210×10^{-4}	2.2191×10^{-4}
Batch size	64	64
Trainable parameters	301,793	521,987
Best validation epoch	10	19

Table 6. Test performance of the validation-selected model trained for each dataset, evaluated at threshold 0.5000.

Dataset	ROC-AUC	Accuracy	Precision	Recall	F1	MCC
Woodbridge benchmark	0.9942	0.9634	0.9730	0.9700	0.9715	0.9203
Synthetic substitution dataset	0.99999	0.9994	0.9992	0.9998	0.9995	0.9987

4.3. Final Test Results

Table 6 reports the primary test metrics, and Table 7 gives the corresponding confusion counts.

Table 7. Test confusion counts at threshold 0.5000 for the models trained on the Woodbridge benchmark and synthetic substitution dataset: true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP).

Dataset	TN	FP	FN	TP
Woodbridge	87,294	4,451	4,961	160,180
Synthetic	91,613	132	25	165,116

The primary operating threshold is fixed at 0.5000 and is not tuned on test predictions. For reference, the validation-selected best-F1 thresholds are 0.3513 for the model trained on the Woodbridge benchmark and 0.9940 for the model trained on the synthetic substitution dataset.

4.4. Qualitative Cross-Attention Analysis

Figure 3 provides a qualitative example from a two-block cross-attention model trained during Woodbridge architecture screening. The example compares sportmedia with a spoof in which the d is replaced by Unicode character U+018C. Although the substitution occurs at only one character, several unchanged characters direct increased attention toward the substituted region. Because these characters still closely match their corresponding characters in the other name, this pattern suggests that the model associates the substituted character with the local alignment shift caused by the substitution.

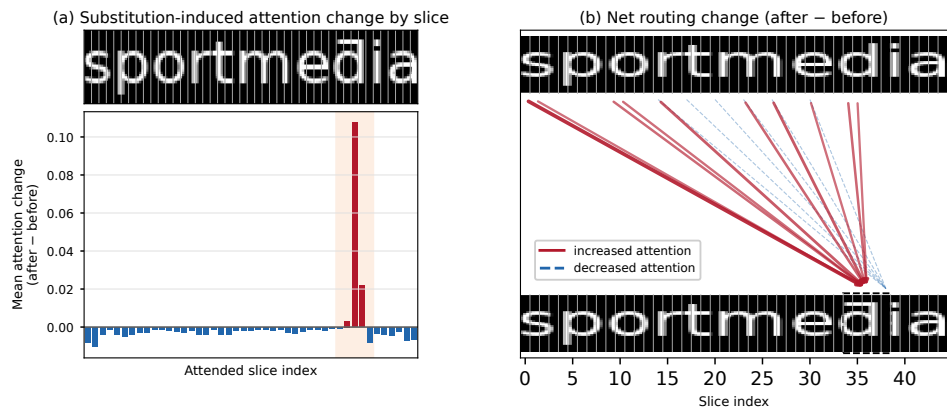


Figure 3. Substitution-induced changes in cross-attention for the Woodbridge pair *sportmedia* and *sportme[U+018C]ia*. The left panel shows the change in mean attention received by each variant-key image slice after subtracting the unchanged-pair baseline; the shaded region denotes slices intersecting the substituted glyph. The right panel shows the strongest signed routing changes involving this region. Red arrows indicate increased attention and blue dashed lines indicate decreased attention following the substitution. Attention weights were extracted from the first block of a Woodbridge two-block cross-attention screening model using 6-pixel slices, stride 3, and two attention heads, and were averaged across those two heads.

4.5. Context Relative to Prior Results

Table 8 summarizes selected results reported in prior work for context. These results were not reproduced under identical conditions: the studies use different datasets, train-test partitions, model-selection procedures, and operating thresholds. They therefore should not be interpreted as direct benchmarks against the test results in Table 6.

Woodbridge et al. report ROC-AUC 0.97 for a Siamese CNN applied to domain-name homoglyph detection [9]. Vinayakumar and Soman report ROC-AUC values between 0.93 and 0.98 for Siamese CNN, long short-term memory, and gated recurrent unit architectures [22]. Although these results were obtained under different datasets and evaluation protocols, the ROC-AUC values achieved by our selected models exceed the prior reported values shown in Table 8. The Woodbridge model reaches 0.9942 ROC-AUC, while the synthetic substitution model reaches 0.99999.

Table 8. Selected prior reported ROC-AUC values for context. Datasets and protocols differ, so these rows are not directly comparable to Table 6.

Method	ROC-AUC	Source
Siamese CNN, domain names	0.9700	[9]
Siamese GRU	0.9800	[22]
Siamese LSTM	0.9700	[22]
Siamese CNN	0.9300	[22]

5. Discussion

Slicing alone does not guarantee improvement.

Within the tested architectures, higher validation scores are associated with contextual sequence processing or direct interaction between the two names rather than slicing alone (the baseline sliced Conv1D family performs worse than the whole-image CNN on both datasets).

The selected pair processor differs across datasets.

Validation-based model selection chooses the Interaction CNN for the Woodbridge benchmark while the same procedure chooses two-block cross-attention for the synthetic

substitution dataset. The difference supports selecting architectures separately for each dataset rather than assuming that one architecture will perform best across both.

Near-saturated validation requires caution and further experimentation.

Several validation scores on the synthetic substitution dataset differ only in the fourth or fifth decimal place. Thus, the final selected models should not be interpreted as the optimal models for each dataset without testing larger search budgets and different random seeds.

Limitations.

The neural experiments use DejaVu Sans at a fixed 32-pixel height, and the OCR atlas also uses DejaVu Sans under fixed raster settings. Robustness to other fonts, rasterizers, resolutions, and display conditions is not established. The visual similarity of Unicode characters is font-dependent. A substitution that is an exact lookalike or OCR-confusable in DejaVu Sans may appear substantially different in another font. Future work should therefore repeat the substitution-generation and evaluation process across multiple fonts.

Final training uses one seed, and the hyperparameter search is not exhaustive; future work will repeat architecture and hyperparameter search across multiple random seeds and broader hyperparameter ranges. Furthermore, the synthetic dataset emphasizes substitutions and does not represent insertions, deletions, or transpositions.

6. Conclusions

This paper presents a rendering-based framework for fine-grained visual homoglyph detection. It also introduces a new synthetic substitution dataset designed to make simple string-distance features less useful by using visually deceptive substitutions and hard negative pairs with similar edit distances.

Names are represented either as ordered vertical-slice sequences or as unsliced images, and seven architecture families are compared using validation-based model selection. The model selected for the Woodbridge benchmark combines a shared Conv1D frontend with an Interaction CNN, while the model selected for the synthetic substitution dataset combines the same shared Conv1D frontend with two bidirectional cross-attention blocks. On their respective test splits, these models achieve ROC-AUC values of 0.9942 and 0.99999.

The architecture comparison also shows that slicing alone is not sufficient: the strongest results come from models that further process or directly compare the visual features of the two names. Broader evaluation across additional fonts, scripts, rendering conditions, and attack types remains an important direction for future work.

Author Contributions: Seton Liu and Hailey Boriel contributed equally to this work. All authors contributed to the study conception and design. Material preparation and analysis were performed by Seton Liu, Hailey Boriel, and Valerie Xin. Amar Gupta supervised the work. All authors have read and agreed to the published version of the manuscript.

Funding: Funding for this research was provided by Itaú Unibanco. The sponsor had no involvement in data collection, modeling decisions, or experimental evaluation.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets used in this study were obtained from and generated using publicly available sources cited in the manuscript. The implementation scripts, trained models, and synthetic dataset are available upon request.

Acknowledgments: During the preparation of this manuscript, the authors used ChatGPT (OpenAI) to assist with grammar and logical consistency review and to suggest revisions to author-written text,

and Codex (OpenAI) to assist with coding and script generation. The authors reviewed and edited all outputs and take full responsibility for the content of this publication.

Conflicts of Interest: The authors have no competing interests.

Disclaimer

Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of Itaú Unibanco, Instituto de Ciência e Tecnologia Itaú, MIT, or other organizations with which the authors are affiliated. This document is not and does not constitute or intend to constitute investment advice or any investment service. It is not and should not be deemed to be an offer to purchase or sell, a solicitation of an offer to purchase or sell, or a recommendation to purchase or sell any securities or other financial instruments. In addition, all data used in this study comply with the Brazilian General Data Protection Law.

Appendix A. Architecture Search Spaces

Tables A1 and A2 summarize the shared and architecture-specific hyperparameters considered during validation-only search.

Table A1. Training and shared-input search space used in the final screening studies.

Hyperparameter	Values or range
Learning rate	Log-uniform from 10^{-4} to 2×10^{-3}
Weight decay	$\{0, 10^{-6}, 10^{-5}, 10^{-4}\}$, categorical
Batch size	$\{32, 64, 128\}$; Interaction CNN used $\{32, 64\}$
Dropout	$\{0.0, 0.1, 0.2\}$
Embedding dimension	$\{64, 128, 256\}$
Strip width	$\{6, 16, 32\}$ for sliced models
Stride	Non-overlapping or 50% overlapping
Outer horizontal padding	Retained or removed
Plain Conv1D stem	Two or three layers; kernel 3 or 5; ReLU or GELU
Other sliced-model stems	Fixed two layers; kernel 3; ReLU
Image height and background	Fixed at 32 pixels; black background
Early-stopping patience	$\{2, 3\}$ epochs
Maximum screening epochs	5
Training and sampling seed	7

Table A2. Architecture-specific search space.

Family	Searched choices
Conv1D	Mean, max, or attention pooling; cosine, calibrated cosine, linear, or MLP pair head
BiLSTM	One or two layers; hidden widths from the configured grid; final-state or masked pooling
Transformer	One or two layers; attention heads from the configured grid; FFN ratios from the configured grid; mean or attention pooling
Cross-attention	Separate one-block and two-block families; heads $\{2, 4, 8\}$; FFN ratio $\{1, 2, 4\}$; ReLU or GELU; mean or attention pooling
Interaction CNN	Cosine or rich four-channel map; one to three Conv2D layers; channels $\{16, 32, 64\}$; kernels $\{3, 5\}$; masked mean, max, or mean-plus-max pooling; linear or one-hidden-layer MLP classifier
Whole-image CNN	Two or three stride-2 Conv2D layers; channels $\{16, 32, 64\}$; kernels $\{3, 5\}$; ReLU or GELU; masked mean, max, or mean-plus-max pooling

1. Anti-Phishing Working Group. *Phishing Activity Trends Report: 1st Quarter 2025*. Available online: https://docs.apwg.org/reports/apwg_trends_report_q1_2025.pdf (accessed on 16 July 2026).

2. Dhamija, R.; Tygar, J.D.; Hearst, M. Why phishing works. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*; ACM, 2006; pp. 581–590. <https://doi.org/10.1145/1124772.1124861>.

3. Roshanbin, N.; Miller, J. Finding homoglyphs: A step towards detecting Unicode-based visual spoofing attacks. In *Web Information System Engineering—WISE 2011*; Bouguettaya, A., Hauswirth, M., Liu, L., Eds.; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2011; Volume 6997, pp. 1–14. https://doi.org/10.1007/978-3-642-24434-6_1.

4. Levenshtein, V.I. Binary codes capable of correcting deletions, insertions and reversals. *Sov. Phys. Dokl.* **1966**, *10*, 707–710. Available online: <https://ui.adsabs.harvard.edu/abs/1966SPhD...10..707L/abstract> (accessed on 16 July 2026).

5. Damerau, F.J. A technique for computer detection and correction of spelling errors. *Commun. ACM* **1964**, *7*, 171–176. <https://doi.org/10.1145/363958.363994>. 477
6. Liu, T.; Zhang, Y.; Shi, J.; Ya, J.; Li, Q.; Guo, L. Towards quantifying visual similarity of domain names for combating typosquatting abuse. In *Proceedings of the 2016 IEEE Military Communications Conference (MILCOM)*; IEEE, 2016; pp. 770–775. <https://doi.org/10.1109/MILCOM.2016.7795422>. 478
7. Mikolov, T.; Sutskever, I.; Chen, K.; Corrado, G.S.; Dean, J. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems* **26**; 2013; pp. 3111–3119. Available online: <https://proceedings.neurips.cc/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html> (accessed on 16 July 2026). 481
8. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*; Association for Computational Linguistics, 2019; Volume 1, pp. 4171–4186. <https://doi.org/10.18653/v1/N19-1423>. 482
9. Woodbridge, J.; Anderson, H.S.; Ahuja, A.; Grant, D. Detecting homoglyph attacks with a Siamese neural network. In *Proceedings of the 2018 IEEE Security and Privacy Workshops (SPW)*; IEEE, 2018; pp. 22–28. <https://doi.org/10.1109/SPW.2018.00012>. 483
10. Gupta, A.; Tomar, L.S.; Garg, R. GlyphNet: Homoglyph domains dataset and detection using attention-based convolutional neural networks. *arXiv* **2023**, arXiv:2306.10392. <https://doi.org/10.48550/arXiv.2306.10392>. 484
11. Ginsberg, A.; Yu, C. Rapid homoglyph prediction and detection. In *Proceedings of the 2018 1st International Conference on Data Intelligence and Security (ICDIS)*; IEEE, 2018; pp. 17–23. <https://doi.org/10.1109/ICDIS.2018.00010>. 485
12. Koch, G.; Zemel, R.; Salakhutdinov, R. Siamese neural networks for one-shot image recognition. In *Proceedings of the ICML 2015 Deep Learning Workshop*; 2015. Available online: <https://www.cs.cmu.edu/~rsalakhu/papers/oneshot1.pdf> (accessed on 16 July 2026). 486
13. Lee, J.S.; Yam, G.P.D.; Chan, J.H. PhishGAN: Data augmentation and identification of homoglyph attacks. *arXiv* **2020**, arXiv:2006.13742. <https://doi.org/10.48550/arXiv.2006.13742>. 487
14. Struppek, L.; Hintersdorf, D.; Friedrich, F.; Brack, M.; Schramowski, P.; Kersting, K. Exploiting cultural biases via homoglyphs in text-to-image synthesis. *J. Artif. Intell. Res.* **2023**, *78*, 1017–1068. <https://doi.org/10.1613/jair.1.15388>. 488
15. Shi, B.; Bai, X.; Yao, C. An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Trans. Pattern Anal. Mach. Intell.* **2017**, *39*, 2298–2304. <https://doi.org/10.1109/TPAMI.2016.2646371>. 489
16. Tatsunami, Y.; Taki, M. Sequencer: Deep LSTM for image classification. In *Advances in Neural Information Processing Systems* **35**; 2022; pp. 38204–38217. Available online: https://proceedings.neurips.cc/paper_files/paper/2022/hash/f9d7d6c695bc983fcfb5b70a5fbd2f-Abstract-Conference.html (accessed on 16 July 2026). 490
17. Pang, L.; Lan, Y.; Guo, J.; Xu, J.; Wan, S.; Cheng, X. Text matching as image recognition. *Proc. AAAI Conf. Artif. Intell.* **2016**, *30*, 2793–2799. <https://doi.org/10.1609/aaai.v30i1.10341>. 491
18. Seth, D.; Stureborg, R.; Pruthi, D.; Dhingra, B. Learning the legibility of visual text perturbations. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*; Association for Computational Linguistics, 2023; pp. 3260–3273. <https://doi.org/10.18653/v1/2023.eacl-main.238>. 492
19. Li, M.; Lv, T.; Chen, J.; Cui, L.; Lu, Y.; Florencio, D.; Zhang, C.; Li, Z.; Wei, F. TrOCR: Transformer-based optical character recognition with pre-trained models. *Proc. AAAI Conf. Artif. Intell.* **2023**, *37*, 13094–13102. <https://doi.org/10.1609/aaai.v37i11.26538>. 493
20. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems* **30**; 2017; pp. 5998–6008. Available online: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html (accessed on 16 July 2026). 494
21. Su, J.; Ahmed, M.H.M.; Lu, Y.; Pan, S.; Bo, W.; Liu, Y. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing* **2024**, *568*, 127063. <https://doi.org/10.1016/j.neucom.2023.127063>. 495
22. Vinayakumar, R.; Soman, K.P. Siamese neural network architecture for homoglyph attacks detection. *ICT Express* **2020**, *6*, 16–19. <https://doi.org/10.1016/j.icte.2019.05.002>. 496

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content. 520